

PLAN DE DESPLIEGUE

PROYECTO: TAG-OK

Historial de Revisiones	3
Introducción	3
Objetivo	3
Arquitectura de Producción	3
Infraestructura Requerida	4
Requisitos Previos	4
Construcción de Imágenes Docker	4
Estrategia de Despliegue	5
Pasos del Despliegue	6
Verificación Post-Despliegue	7
Monitoreo y Mantenimiento	7
Resolución de Problemas Comunes	8

Historial de Revisiones

Fecha	Versión	Descripción	Autor
20 de Junio de 2026	1.0	Versión inicial del documento	Diego Rodríguez, Paulina Troncoso, Ricardo Sánchez

Introducción

El presente documento describe la estrategia y los pasos necesarios para desplegar el sistema TAG OK desde el entorno de desarrollo hacia un entorno productivo. El objetivo es establecer un procedimiento claro y repetible que permita poner el sistema a disposición de usuarios reales, específicamente conductores de la Región Metropolitana de Santiago y administradores de Grupo Sentte.

El sistema se compone de múltiples servicios coordinados mediante Docker Compose: un API Gateway como punto de entrada único, un servicio de rutas y tarifas, un servicio de historial, bases de datos PostgreSQL con PostGIS y MongoDB, un broker de eventos Kafka, y un panel web administrativo servido mediante Nginx. Todos los servicios Java comparten una estructura de construcción en dos etapas, mientras que el frontend web utiliza una estrategia similar con Node.js y Nginx.

Objetivo

Este plan tiene como objetivo definir la estrategia de despliegue del ecosistema TAG OK en un entorno productivo, detallando la infraestructura necesaria, la construcción de imágenes Docker, la configuración de servicios, los pasos secuenciales para el despliegue y las pruebas de verificación post-despliegue. Está dirigido al equipo de desarrollo, al cliente Grupo Sentte y a los evaluadores del proyecto académico.

Arquitectura de Producción

El sistema se despliega mediante Docker Compose sobre una red interna llamada tag-ok que interconecta todos los servicios. El punto de entrada para los clientes es el gateway-service, expuesto en el puerto 8080, que enruta las peticiones a los servicios internos correspondientes. El panel web administrativo se sirve mediante el contenedor frontend-tag en el puerto 80.

La capa de datos se compone de PostgreSQL con las extensiones PostGIS y pgRouting para el servicio de rutas, utilizando la imagen pgrouting/pgrouting:15-3.3-3.4.1, y MongoDB en su versión 7 para el servicio de historial. Ambos servicios de base de datos cuentan con

interfaces de administración web: pgAdmin en el puerto 1000 y Mongo Express en el puerto 8002.

La comunicación asíncrona entre servicios se realiza mediante Apache Kafka en su versión 7.4.0 con Zookeeper. Las variables de entorno sensibles se gestionan mediante un archivo .env que no se incluye en el repositorio.

Infraestructura Requerida

El despliegue requiere un servidor o máquina virtual con sistema operativo Linux, con un mínimo de 4 núcleos de CPU, 8 GB de RAM y 30 GB de almacenamiento disponible. La plataforma debe tener Docker Engine y Docker Compose instalados, así como acceso a internet para la descarga de imágenes base y la conexión con servicios externos como Supabase y Gemini.

Se requiere un nombre de dominio o dirección IP pública para exponer el gateway-service en el puerto 8080 y el panel web en el puerto 80. Para entornos productivos reales se recomienda configurar certificados SSL mediante un proxy inverso como Nginx o Traefik, aunque esto queda fuera del alcance del MVP actual.

Requisitos Previos

Antes de iniciar el despliegue, se debe preparar el servidor con Docker Engine y Docker Compose instalados. Se debe clonar el repositorio del proyecto en el servidor y preparar el archivo .env en la carpeta Producto con todas las variables de entorno necesarias.

Las variables requeridas incluyen SERVER_PORT con el valor 8080 para el puerto del gateway, SUPABASE_URL con la URL del proyecto en Supabase, SUPABASE_ANON_KEY con la clave anónima de Supabase, y SUPABASE_JWT_ISSUER_URI con la URL de emisión de tokens JWT. Para el servicio de historial se requiere GEMINI_API_KEY con la clave de Gemini y GEMINI_MODEL con el modelo a utilizar, además de MONGODB_URI con la cadena de conexión a MongoDB. Para Kafka se requiere KAFKA_BOOTSTRAP_SERVERS con la dirección del broker. Para PostgreSQL se requieren POSTGRES_DB, POSTGRES_USER y POSTGRES_PASSWORD.

Adicionalmente, se debe haber ejecutado el esquema SQL de Supabase en el proyecto correspondiente y tener disponibles los archivos JSON de pódicos para su carga inicial.

Construcción de Imágenes Docker

Los servicios Java del sistema comparten una estructura de construcción en dos etapas. La primera etapa utiliza la imagen base maven:3.9.9-eclipse-temurin-21 para compilar el proyecto. Se copia el código fuente completo al contenedor y se ejecuta el comando mvn clean package -DskipTests, que genera el archivo JAR ejecutable sin ejecutar las pruebas

unitarias. La segunda etapa utiliza la imagen base eclipse-temurin:21-jdk-jammy como entorno de ejecución, que es más ligera y contiene únicamente el JDK 21 necesario para ejecutar la aplicación. Se copia el archivo JAR generado en la etapa anterior y se configura como punto de entrada con el comando `java -jar app.jar`. El puerto 8080 se expone para la comunicación.

Los servicios que utilizan esta estructura son el gateway-service, el routes-service y el history-service. Cada uno cuenta con su propio Dockerfile en su respectiva carpeta, y el contexto de construcción se limita a dicha carpeta según la configuración del `docker-compose.yml`.

El frontend web utiliza una construcción también en dos etapas. La primera etapa emplea la imagen base `node:20` para construir la aplicación. Se copian los archivos `package.json` y `package-lock.json`, se ejecuta `npm install` para descargar las dependencias, luego se copia el resto del código fuente y se ejecuta `npm run build` para generar los archivos estáticos en la carpeta `dist`. La segunda etapa utiliza la imagen `nginx:alpine`, que es extremadamente ligera y eficiente para servir contenido estático. Se copian los archivos generados en la etapa anterior a la carpeta `/usr/share/nginx/html` de Nginx y se expone el puerto 80 para el acceso web.

Los servicios de infraestructura como PostgreSQL, MongoDB, Kafka y Zookeeper utilizan imágenes preconstruidas desde Docker Hub, por lo que no requieren un proceso de build personalizado.

Estrategia de Despliegue

El despliegue se realiza mediante Docker Compose, que orquesta todos los servicios definidos en el archivo `docker-compose.yml`. La estrategia consiste en levantar primero los servicios de infraestructura base que no requieren construcción, luego construir y levantar los servicios de aplicación backend, y finalmente construir y levantar el frontend web.

El orden de despliegue es el siguiente. Primero se levantan las bases de datos: db-rutas con PostgreSQL y PostGIS, y db-historial con MongoDB. Luego se levantan las interfaces de administración: pgadmin y mongo-express. A continuación se levanta el broker de eventos: zookeeper-tag, kafka-tag y kafka-setup-tag para la inicialización de tópicos. Después se construyen y levantan los servicios de aplicación: routes-service y history-service. Luego se construye y levanta el punto de entrada: gateway-service. Finalmente se construye y levanta el panel web: frontend-tag.

Durante la primera ejecución, Docker Compose construirá las imágenes de los servicios que tienen definida la directiva `build` en el archivo `docker-compose.yml`. Las imágenes resultantes quedarán almacenadas en el registro local de Docker del servidor. En ejecuciones posteriores, si no hay cambios en el código fuente, se puede utilizar la bandera `--no-build` para evitar reconstruir las imágenes.

Para la aplicación móvil Android, el despliegue se realiza mediante la generación de un archivo APK o AAB desde Android Studio, configurando las URLs de producción para que apunten al gateway-service desplegado en el servidor.

Pasos del Despliegue

El primer paso es preparar el servidor. Se instala Docker Engine y Docker Compose siguiendo la documentación oficial para el sistema operativo del servidor. Se verifica la instalación con los comandos `docker --version` y `docker compose version`, que deben mostrar las versiones instaladas.

El segundo paso es clonar el repositorio del proyecto en el servidor mediante Git, navegando hasta la carpeta Producto donde se encuentra el archivo `docker-compose.yml` principal.

El tercer paso es configurar las variables de entorno. Se crea el archivo `.env` en la carpeta Producto con todas las variables requeridas, reemplazando los valores de ejemplo por las credenciales reales de producción para Supabase, Gemini, MongoDB y Kafka.

El cuarto paso es inicializar las bases de datos. Se ejecuta el comando `docker compose up -d db-rutas db-historial pgadmin mongo-express` para levantar los servicios de persistencia y sus interfaces de administración. Se espera a que estén saludables verificando con `docker ps`.

El quinto paso es levantar los servicios de mensajería. Se ejecuta `docker compose up -d zookeeper-tag kafka-tag kafka-setup-tag`. El contenedor `kafka-setup-tag` se encarga de crear los tópicos iniciales necesarios para la comunicación entre servicios y luego se detiene automáticamente una vez completada su tarea.

El sexto paso es construir y levantar los servicios de aplicación. Se ejecuta `docker compose up -d routes-service history-service`. Docker Compose construirá las imágenes utilizando los Dockerfile de cada servicio. El proceso de build de cada servicio Java incluye la descarga de dependencias Maven y la compilación del proyecto, lo que puede tomar varios minutos la primera vez. En ejecuciones posteriores, la caché de capas de Docker acelerará significativamente el proceso.

El séptimo paso es construir y levantar el gateway. Se ejecuta `docker compose up -d gateway-service`. Este servicio se construye con la misma estructura de dos etapas que los demás servicios Java y expone el puerto 8080 como punto de entrada único al sistema.

El octavo paso es construir y levantar el frontend web. Se ejecuta `docker compose up -d frontend-tag`. Docker Compose construirá la imagen utilizando el Dockerfile del frontend, que compila la aplicación React con Node.js y la sirve con Nginx. El panel web queda expuesto en el puerto 80.

El noveno paso es la carga de datos iniciales. Se cargan las calles de Santiago mediante la herramienta `osm-importer`, ejecutando los comandos de compilación y ejecución desde la

carpeta correspondiente. Se cargan los pórticos de autopistas mediante el panel de administración web, accediendo a la funcionalidad de carga masiva y seleccionando los archivos JSON de la carpeta porticos.

Para la aplicación móvil, se genera el archivo APK desde Android Studio en modo release, configurando la URL del gateway-service de producción, y se distribuye a los usuarios mediante el canal acordado con el cliente.

Verificación Post-Despliegue

Una vez completado el despliegue, se debe verificar que todos los contenedores estén en ejecución con el comando `docker ps`. Deben aparecer todos los servicios: `db-rutas`, `pgadmin_container`, `db-historial`, `mongo_express`, `zookeeper-tag`, `kafka-tag`, `routes-service`, `history-service`, `gateway-service` y `frontend-tag`.

Se verifica la documentación de la API accediendo a `http://servidor:8080/swagger-ui/index.html`, que debe mostrar todos los endpoints organizados por servicio, confirmando que el gateway está enrutando correctamente las peticiones.

Se verifica el panel web accediendo a `http://servidor`, que debe mostrar la página de inicio de sesión servida por Nginx. Se realiza una prueba de autenticación con credenciales válidas de Supabase para confirmar la correcta integración.

Se verifica la funcionalidad principal realizando un cálculo de ruta de prueba entre dos puntos de Santiago a través de la API o el panel web, confirmando que el sistema devuelve los pórticos cruzados y el costo estimado según el tipo de vehículo y el calendario tarifario.

Se verifica la aplicación móvil instalando el APK generado en un dispositivo físico o emulador, comprobando la conexión con el gateway-service, la autenticación con Supabase, y la funcionalidad de planificación de viajes con cálculo de costo.

Monitoreo y Mantenimiento

Para el monitoreo básico del sistema se utilizan los logs de Docker. El comando `docker compose logs -f` permite ver los logs de todos los servicios en tiempo real. Para ver los logs de un servicio específico se utiliza `docker compose logs -f nombre-servicio`. Esto es útil para identificar errores en tiempo de ejecución o verificar el procesamiento de peticiones.

Los datos persistentes se almacenan en volúmenes de Docker definidos en el archivo `docker-compose.yml`: `postgres_data` para los datos de PostgreSQL, `pgadmin_data` para la configuración de pgAdmin, y `mongo_data` para los datos de MongoDB. Estos volúmenes conservan los datos incluso si los contenedores se detienen o eliminan, siempre que no se utilice la bandera `-v` al ejecutar `docker compose down`.

Se recomienda realizar respaldos periódicos de las bases de datos. Para PostgreSQL se utiliza `pg_dump` ejecutado desde el contenedor `db-rutas`. Para MongoDB se utiliza `mongodump` ejecutado desde el contenedor `db-historial`. La frecuencia de los respaldos dependerá del volumen de datos y los requisitos del cliente.

Para actualizar los servicios a una nueva versión, se debe obtener la última versión del código fuente mediante Git, reconstruir las imágenes con `docker compose build`, y recrear los contenedores con `docker compose up -d`. Los volúmenes de datos no se verán afectados por este proceso.

Para detener el sistema sin perder datos se ejecuta `docker compose stop`. Para detener y eliminar los contenedores conservando los datos se ejecuta `docker compose down`. Para eliminar todo incluidos los datos y empezar desde cero se ejecuta `docker compose down -v`.

Resolución de Problemas Comunes

Si algún contenedor no inicia, se revisan los logs con `docker compose logs nombre-servicio` para identificar la causa. Los problemas más comunes incluyen variables de entorno faltantes o mal escritas, puertos ya ocupados en el servidor, o dependencias entre servicios no satisfechas por el orden de inicio.

Si la construcción de imágenes Java falla con errores de compilación, se debe verificar que el código fuente esté completo y que no haya errores de sintaxis. También se debe verificar la conectividad a internet durante la etapa de build, ya que Maven necesita descargar dependencias.

Si la construcción del frontend falla, se debe verificar que el archivo `package.json` esté completo y que `npm install` pueda descargar todas las dependencias. Errores comunes incluyen problemas de conectividad o versiones incompatibles de paquetes.

Si el `gateway-service` no puede comunicarse con `routes-service` o `history-service`, se verifica que ambos servicios estén en ejecución con `docker ps` y que los nombres de host en la red interna `tag-ok` coincidan con los configurados en el gateway. La resolución de nombres entre contenedores en la misma red de Docker Compose es automática.

Si Kafka no inicia correctamente, se verifica que Zookeeper esté completamente operativo antes que Kafka. El contenedor `kafka-tag` depende de `zookeeper-tag` y espera a que esté disponible. También se debe verificar que el archivo de propiedades del servidor Kafka esté correctamente montado como volumen.

Si el panel web carga pero no muestra datos, se verifica que la URL del gateway configurada en el archivo `src/api/axios.ts` del frontend apunte correctamente al `gateway-service` en producción. Si se despliega en un servidor remoto, se debe usar la dirección IP o dominio del servidor en lugar de `localhost`.

Si la autenticación con Supabase falla en producción, se verifica que las variables `SUPABASE_URL`, `SUPABASE_ANON_KEY` y `SUPABASE_JWT_ISSUER_URI` estén

correctamente configuradas en el archivo .env del entorno de producción y que el proyecto de Supabase tenga habilitados los proveedores de autenticación necesarios.

Si la carga de pórticos falla desde el panel web, se verifica que los archivos JSON tengan el formato correcto y que el servicio routes-service esté en ejecución y accesible desde el gateway-service. También se puede utilizar curl o Postman para cargar los pórticos directamente a través de la API como alternativa.